



L OVELY
P ROFESSIONAL
U NIVERSITY

SMART BANKING AI SYSTEM WITH ANTISPOOF

NAME: R JAI HARI BALAJI

REGISTRATION NO.: 12305375

ROLL NO.: 01

NAME: K PRASANNA

REGISTRATION NO.: 12324215

ROLL NO.: 10

NAME: A ADHITHYAA

REGISTRATION NO: 12305845

ROLL NO.: 07

COURSE CODE: ECE363

COURSE CODE: CREATING AN IOT DEVICE

TABLE OF CONTENTS

S No.	CONTENTS
1.	ABSTRACT
2.	INTRODUCTION
3.	OBJECTIVES
4.	COMPONENTS USED
5.	SYSTEM ARCHITECTURE
6.	SOFTWARE REUQUIREMENTS
7.	WORK FLOW OF THE SYSTEM
8.	SPOOF VERIFICATION
9.	OTP VERIFICATION
10.	WORKING OF SYSTEM
11.	RESULTS
12.	CONCLUSION

ABSTRACT

The Banking AI System is a smart banking security project developed using artificial intelligence facial recognition and OTP verification. The system identifies a customer using live face recognition through a camera and provides secure access only after successful OTP verification. The main goal of the project is to improve banking security and reduce unauthorized access using real time AI based authentication.

The system uses a Raspberry Pi camera for capturing the customer face and sends the live stream to the server. The server performs facial recognition using machine learning models and identifies the customer from the database. After successful recognition the system automatically generates an OTP and sends it to the registered mobile number of the customer using SMS service. The customer enters the OTP and if the OTP matches the stored value the banking dashboard is displayed.

The project also includes spoof detection and session banning features. If fake access or spoofing is detected the system blocks the session and prevents OTP generation. Customer login details are stored in CSV and database files for validation and security analysis.

The system is developed using Python Flask Java InsightFace SQLite and Raspberry Pi. This project provides a secure intelligent and modern banking authentication solution using AI based technologies.

INTRODUCTION

In recent years banking systems have rapidly shifted towards digital platforms where security and user authentication play a major role. Traditional authentication methods such as passwords PIN numbers and card based verification are vulnerable to security threats like password theft phishing unauthorized access and identity fraud. Due to the increasing number of cyber attacks modern banking systems require stronger and more intelligent authentication techniques to protect customer information and financial transactions.

This project presents an AI Powered Banking Authentication System that uses facial recognition and OTP verification for secure customer login. The system uses artificial intelligence to recognize customers through live video captured using a camera connected to a Raspberry Pi. The live video stream is processed by a machine learning model which detects the face and compares it with stored customer data available in the database. After successful recognition the system generates a one time password and sends it to the registered mobile number of the customer. The customer gains access to the banking dashboard only after entering the correct OTP.

The project also includes spoof detection and session blocking features to improve security. If any fake access attempt or spoofing activity is detected the system blocks the session and prevents unauthorized login. The system is developed using Python Flask InsightFace Java Twilio SQLite and Raspberry Pi technologies.

The main aim of the project is to create a secure intelligent and user friendly banking authentication system using artificial intelligence and real time verification techniques.

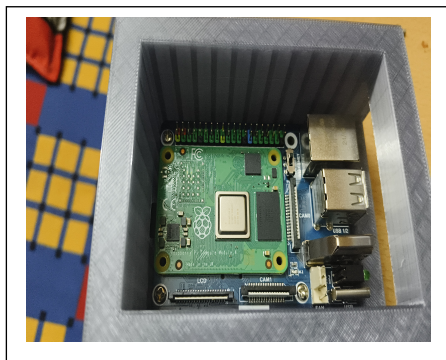
OBJECTIVES

The main objective of this project is to develop a secure and intelligent banking authentication system using artificial intelligence and real time facial recognition. The system is designed to improve customer security by replacing traditional password based authentication with AI based face verification and OTP validation. Another important objective is to provide multi layer authentication where the customer is verified both through facial recognition and mobile OTP before accessing the banking dashboard. The project also aims to prevent unauthorized access and fake login attempts using spoof detection and session banning techniques. Real time processing and low latency authentication are additional goals of the system to ensure smooth customer experience. The project further focuses on maintaining customer records login logs and verification details using database and CSV storage methods. Overall the objective of this system is to create a reliable secure and modern banking authentication platform using machine learning and intelligent automation.

COMPONENTS USED

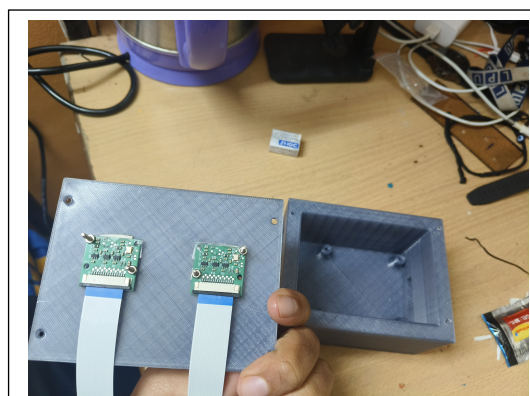
1. Compute Module 4

Compact and powerful processing unit handling dual-camera input, compression, and high-speed data transfer to the server via Ethernet reliably.



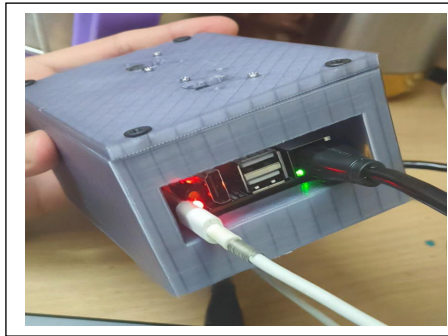
2. Dual CSI Camera

Two high-resolution CSI cameras capturing synchronized frames, providing stereo visual input for accurate face detection and reliable spoof identification.

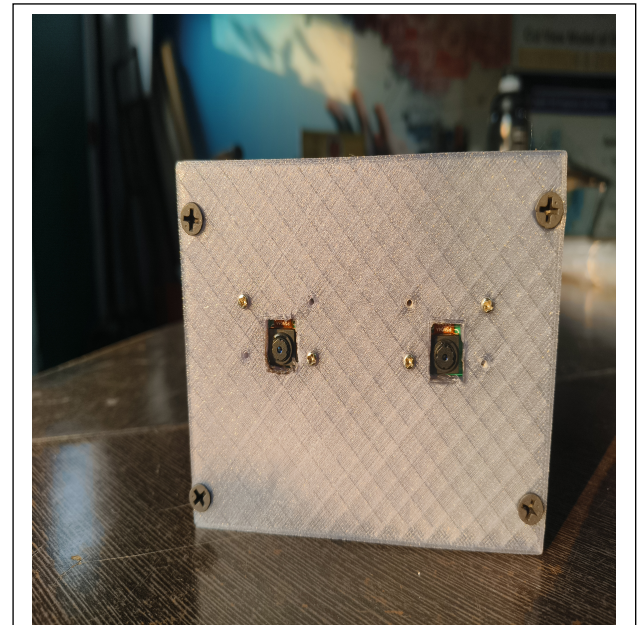


3. Ethernet Cable

Ensures fast, low-latency, uninterrupted communication between CM4 and server, supporting real-time streaming even in offline or unstable network environments.



SYSTEM ARCHITECTURE



The system consists of five major modules:

User Interface

The customer accesses the banking system through a Flask web application.

Facial Recognition Module

The ML model detects and recognizes the customer face using InsightFace embeddings.

OTP Generation Module

The system generates a random six digit OTP after successful recognition.

SMS Delivery Module

The OTP is sent to the customer mobile number using Twilio SMS API through Java.

Verification and Dashboard Module

The OTP entered by the customer is validated and the banking dashboard is displayed.

SOFTWARE REQUIREMENTS

- **Python**

Python is the main programming language used for developing the project. It is used for backend processing facial recognition OTP generation database handling and server side operations. Python provides easy integration with machine learning and web development libraries which makes it suitable for AI based applications.

- **Flask**

Flask is a lightweight Python web framework used to create the banking web application. It handles webpage routing OTP verification customer dashboard display and communication between the frontend and backend modules. Flask provides fast and simple web application development.

- **Java**

Java is used for sending OTP messages to customer mobile numbers through the Twilio SMS API. The Java program establishes a secure connection with the Twilio server and dynamically sends OTP messages generated by the Flask application.

- **InsightFace**

InsightFace is the machine learning framework used for facial recognition. It extracts facial embeddings from live video frames and compares them with stored customer embeddings for identity verification. It provides high accuracy and real time face recognition performance.

- **OpenCV**

OpenCV is used for image and video processing operations. It captures video frames from the camera processes facial images and supports real time computer vision tasks within the system.

- **SQLite**

SQLite is used as the database management system for storing customer details login information and verification records. It is lightweight simple and suitable for local database applications.

- **Twilio API**

Twilio API is used for sending OTP messages to customer mobile numbers. It allows the system to perform secure SMS based verification during customer authentication.

- **HTML CSS JavaScript**

HTML CSS and JavaScript are used for developing the frontend webpages of the banking system. HTML creates the webpage structure CSS improves the design and appearance while JavaScript handles button actions and webpage interactions.

WORK FLOW OF THE SYSTEM

The Banking AI System works by combining facial recognition OTP verification and spoof detection to provide secure customer authentication. The complete system operates in multiple stages where each module performs a specific task for improving banking security and preventing unauthorized access.

Initially the customer opens the banking web application through the Flask server. The login page contains a Start Recognition button which begins the authentication process. When the button is pressed the Flask application launches the machine learning based banking recognition system called `banking_ml.py`. This file acts as the core AI engine of the project.

The Raspberry Pi camera starts capturing live video frames of the customer standing in front of the camera. The live video stream is transmitted to the server using Ethernet communication and MJPEG frame transfer. The server continuously receives video frames and processes them in real time. The system uses the InsightFace model for facial recognition. The model first detects the human face from the incoming video frame and extracts unique facial embeddings from the detected face. Facial embeddings are mathematical feature vectors that represent the unique structure of a person face. These embeddings are then compared with the customer embeddings stored in the database.

The cosine similarity algorithm is used to compare the live embedding with the stored embeddings. If the similarity score crosses the predefined threshold the customer is identified successfully. The recognized customer name is then written into a recognition file called `recognized_customer.txt` which acts as a communication bridge between the machine learning module and the Flask application.

Before allowing authentication the system also performs spoof detection. The project uses a YOLO based object detection model to identify suspicious objects such as mobile phones or spoofing attempts. If spoofing is detected the system immediately blocks the customer session and prevents OTP generation. This session banning mechanism improves banking security and prevents fake login attempts.

After successful customer recognition the Flask application reads the recognized customer name from the recognition file. Using the customer name the system retrieves customer details such as phone number email account details and KYC information from the customer database.

The system then generates a random six digit OTP using Python random functions. This OTP is temporarily stored in memory for validation purposes. The OTP is sent to the customer registered mobile number using Twilio SMS service through a Java based SMS module.

The customer receives the OTP on the registered mobile number and enters it on the OTP verification page. The Flask backend compares the entered OTP with the stored OTP value. If both values match successfully the customer authentication is completed.

After successful verification the banking dashboard is displayed. The dashboard contains customer information such as account number balance branch details loan status and KYC status. At the same time the system stores customer login details such as customer name login time and login date into SQLite database and CSV log files for future validation and monitoring.

Finally the recognized customer file and OTP data are cleared to prevent repeated authentication issues and stale session problems. This ensures that every login session starts with fresh facial recognition and secure OTP validation. The entire process works in real time and provides a secure intelligent and user friendly banking authentication system using artificial intelligence and machine learning technologies.

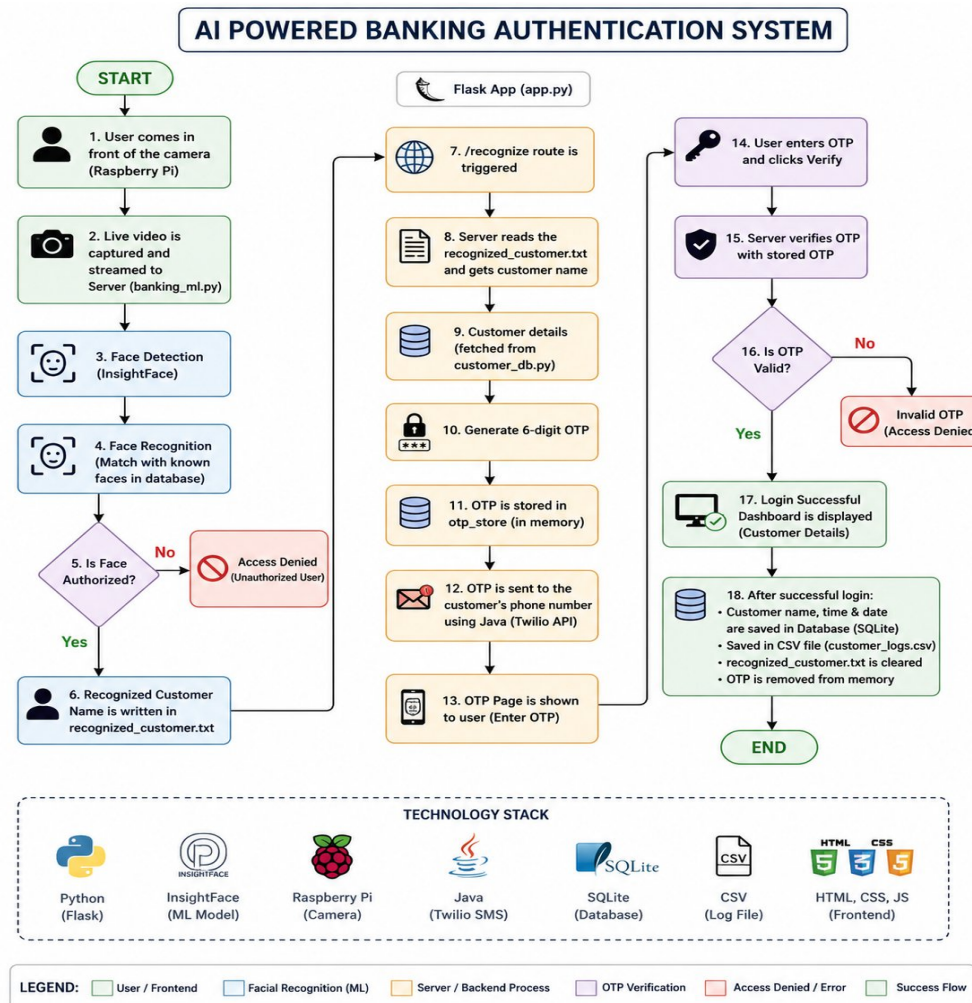


Figure: System architecture flow

SPOOF DETECTION

Spoof detection is one of the most important security features implemented in the Banking AI System. The main purpose of spoof detection is to prevent unauthorized users from gaining access to the banking dashboard using fake methods such as mobile phone images printed photographs or recorded videos of a customer face. In many traditional facial recognition systems attackers can bypass authentication by simply showing a customer photo in front of the camera. To overcome this security issue the project includes a real time spoof detection mechanism.

The system continuously monitors the incoming live video stream while performing facial recognition. Along with the facial recognition model the project also uses a YOLO based object detection model for identifying suspicious objects and spoofing activities. The object detection model is trained to detect mobile phones and similar spoof related objects that may be used for fake authentication attempts.

During the authentication process the system checks whether a detected face is associated with suspicious spoof objects. If the system identifies a possible spoof attempt it immediately marks the customer session as suspicious. The system then

increases the spoof count for that customer and verifies the spoof condition across multiple consecutive frames to avoid false detection.

Once the spoof condition is confirmed the system performs session banning. The recognized customer session is removed and the customer is temporarily blocked from authentication. OTP generation is also stopped immediately to prevent unauthorized access. A session banned message is displayed in the server terminal indicating that a spoof attempt has been detected.

The spoof detection mechanism improves the reliability and security of the banking authentication system. It ensures that only genuine live customers are allowed to complete the authentication process. This feature makes the project more secure compared to traditional facial recognition systems that do not include spoof prevention techniques.



Figure: Spoof attempt with phone

OTP VERIFICATION

OTP verification acts as the second layer of security in the Banking AI System. Even after successful facial recognition the customer is not allowed to access the banking dashboard directly. The system performs additional verification using a One Time Password to ensure that the recognized customer is the genuine account holder. This multi layer authentication process improves banking security and reduces the possibility of unauthorized access.

After successful customer recognition the Flask application retrieves the registered mobile number of the customer from the customer database. The system then generates a random six digit OTP using Python random functions. This OTP is unique for every login session and remains valid only for a short duration during the verification process. The generated OTP is temporarily stored inside the backend memory using the OTP storage module. After storing the OTP the system sends the OTP message to the registered customer mobile number using Twilio SMS API. A Java based SMS module is used for establishing communication with the Twilio server and dynamically sending the OTP message.

The customer receives the OTP on the registered mobile device and enters it on the OTP verification webpage. The Flask backend receives the entered OTP and compares it with the stored OTP value available in the backend memory. If both OTP values match correctly the customer authentication process is considered successful. Once the OTP verification is completed the banking dashboard is displayed. The dashboard contains customer information such as account details balance KYC status and loan details. At the same time the customer login details are stored in CSV and database files for security monitoring and validation purposes. If the entered OTP is incorrect the system immediately denies access and displays an Invalid OTP message. The customer is required to repeat the authentication process for secure access. After successful verification the stored OTP and recognized customer file are cleared automatically to prevent repeated login issues and stale session problems. This OTP verification process provides an additional security layer and strengthens the overall banking authentication system.

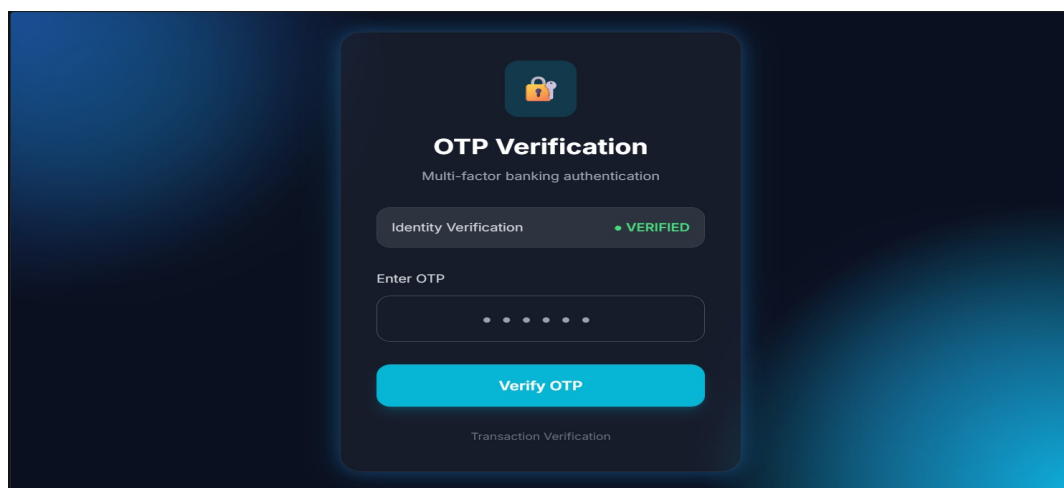


Figure: OTP verification interface

WORKING OF THE SYSTEM

The Banking AI System works by combining artificial intelligence facial recognition OTP verification and secure customer validation to provide protected banking access. The system starts when the customer opens the banking web application developed using Flask. The login page contains a Start Recognition button which begins the authentication process.

When the customer clicks the Start Recognition button the Flask backend automatically launches the machine learning module called `banking_ml.py`. At the same time the Raspberry Pi camera starts capturing the live video stream of the customer standing in front of the camera. The live video frames are continuously transmitted to the server using Ethernet based communication.

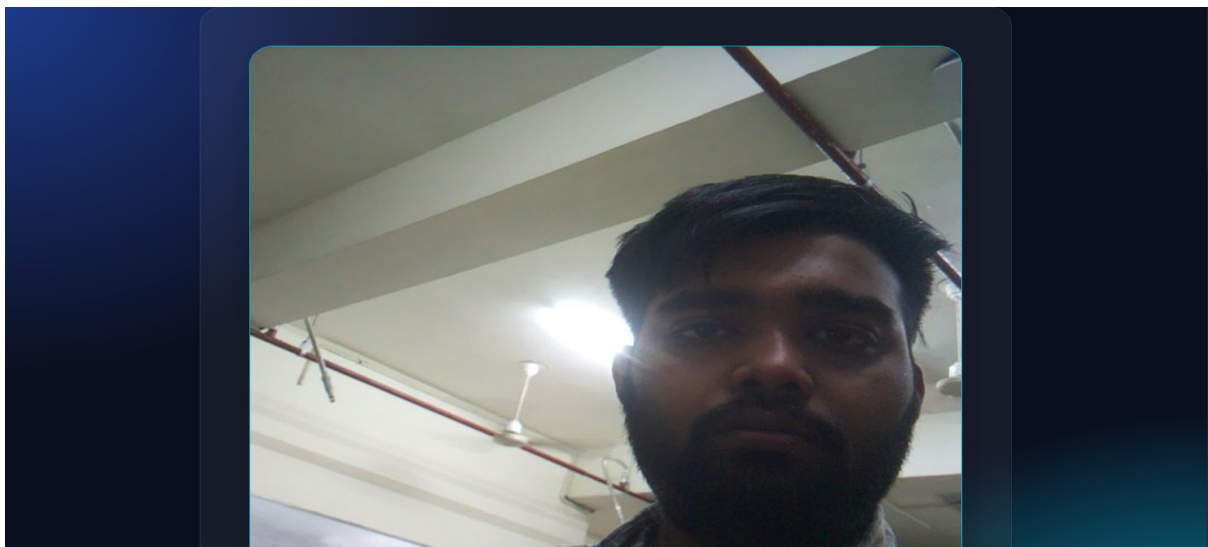
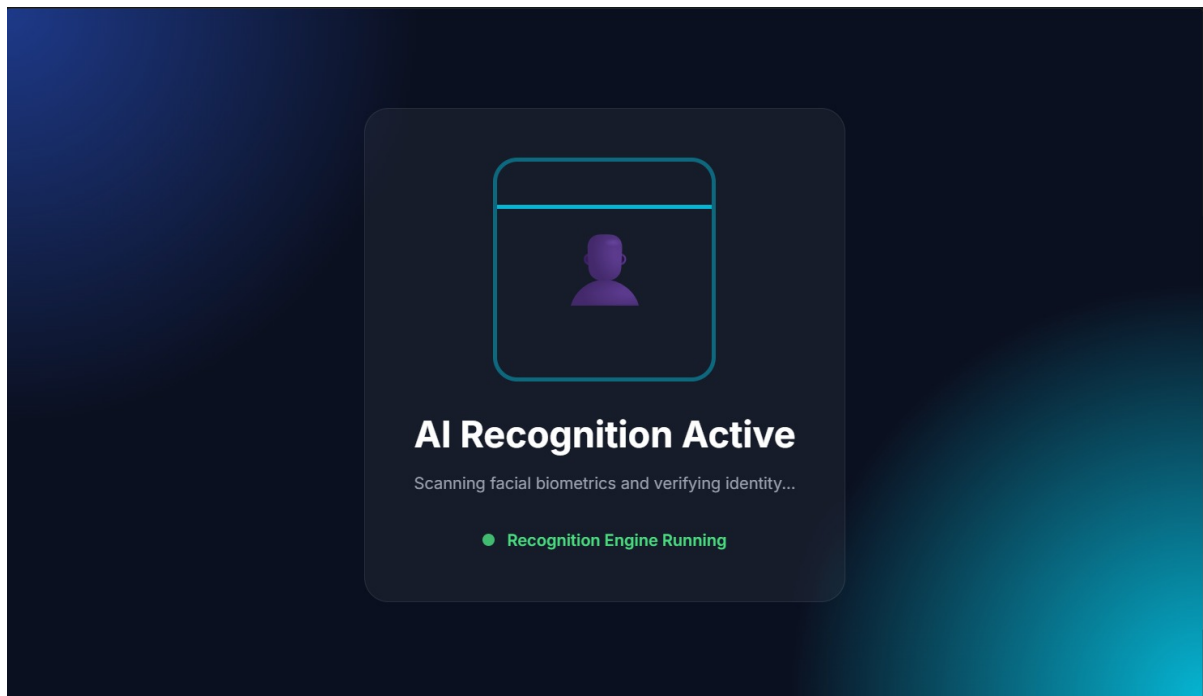


Figure: User Verification through facial recognition

The machine learning model processes each incoming video frame in real time. The InsightFace model detects the customer face and extracts unique facial embeddings from the captured image. These embeddings are mathematical facial features that uniquely represent the customer identity. The extracted embeddings are then compared with the embeddings stored in the customer database.

If the similarity score crosses the recognition threshold the customer is identified successfully. The recognized customer name is written into the `recognized_customer.txt` file which acts as a communication bridge between the ML module and the Flask server. During this stage the system also performs spoof detection. The YOLO based object detection model continuously monitors the video frames for suspicious objects such as mobile phones fake images or spoof

attempts. If spoofing is detected the session is banned immediately and OTP generation is stopped to prevent unauthorized access.

After successful recognition the Flask server reads the recognized customer name and retrieves the corresponding customer details from `customer_db.py`. The system then generates a random six digit OTP using Python random functions. The generated OTP is stored temporarily in memory using the `otp_store` module.

Next the OTP is sent to the registered mobile number of the customer using Twilio SMS service through the Java based `SendSms` module. The customer receives the OTP on the mobile device and enters it on the OTP verification webpage displayed by the Flask application.

The entered OTP is compared with the stored OTP available in the backend memory. If the OTP matches successfully the customer authentication is completed and the banking dashboard is displayed. The dashboard contains important customer information such as customer name account number balance KYC status branch details and loan information.

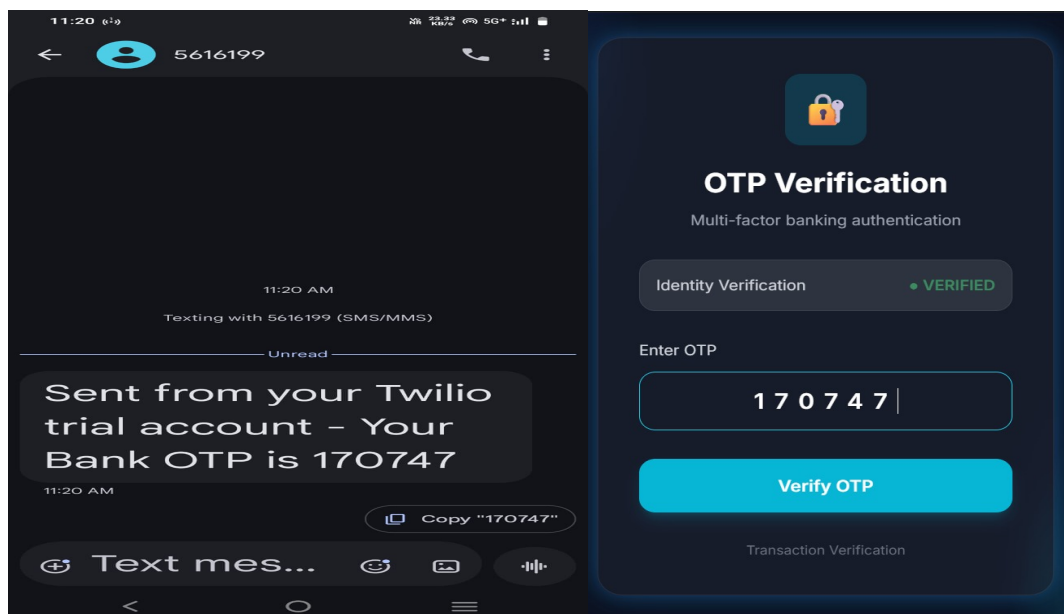


Figure: User Receiving OTP to their registered phone number through twilio API

After successful login the system stores customer login details in CSV log files and database storage for validation and security monitoring. Finally the recognized customer file and OTP data are cleared automatically to prevent stale sessions and repeated authentication problems. This complete process provides a secure intelligent and real time AI based banking authentication system.

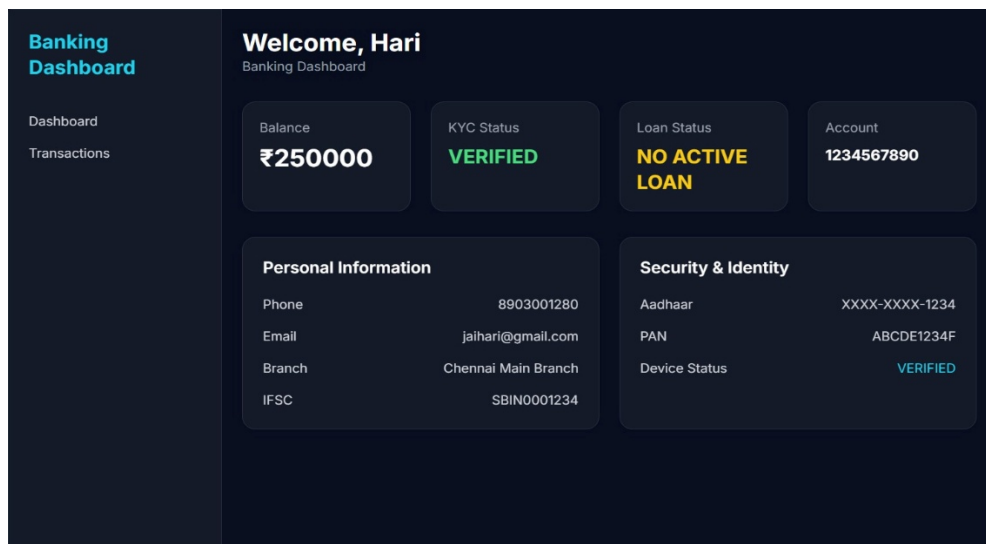


Figure: Dashboard Containing user's bank details and other information

RESULTS

The Banking AI System was successfully developed and tested for secure customer authentication using facial recognition and OTP verification. The system performed real time customer identification accurately through live video streaming using Raspberry Pi and machine learning models. The facial recognition module was able to identify registered customers from the stored embedding database with good accuracy and low latency.

The system successfully generated and delivered OTP messages to registered customer mobile numbers using Twilio SMS service. OTP verification was completed successfully and customers were able to access the banking dashboard only after valid authentication. The dashboard displayed customer information such as account balance KYC status account details and branch information correctly.

The spoof detection module also worked successfully during testing. When fake access attempts were performed using mobile phones or suspicious objects the system detected the spoof condition and immediately banned the customer session. OTP generation was stopped during spoof detection which improved overall banking security.

Customer login details were stored correctly in SQLite database and CSV log files after successful authentication. The system also cleared previous recognition data and OTP values after verification which prevented stale session problems and repeated authentication issues.

The project demonstrated stable performance in real time conditions and provided secure multi layer authentication using facial recognition spoof detection and OTP verification. The final output confirmed that the system can be used as an intelligent and secure banking authentication solution using artificial intelligence and machine learning technologies.

CONCLUSION

The Banking AI System successfully achieved secure customer authentication using artificial intelligence facial recognition and OTP verification techniques. The project demonstrated how

machine learning can be integrated with banking applications to improve security and reduce unauthorized access. The system was able to recognize customers in real time using live video streaming and provide secure dashboard access only after successful OTP verification.

The project also included spoof detection and session banning mechanisms which improved the reliability of the authentication process. Fake access attempts using suspicious objects or spoofing methods were detected successfully and blocked immediately. This increased the overall security of the banking system and prevented unauthorized login attempts.

The integration of Raspberry Pi Flask InsightFace Twilio and SQLite helped in creating a complete intelligent banking authentication platform. The system maintained customer login records and verification logs properly using database and CSV storage methods. Real time performance low latency recognition and smooth OTP verification were achieved successfully during testing.

The project proved that AI based authentication systems can provide better security compared to traditional password based banking systems. The developed system is secure intelligent user friendly and suitable for modern banking applications. Future improvements can further enhance the system by adding advanced anti spoofing techniques cloud integration and mobile banking support.